Replicated Counter Service Test Report

Assessment: Building and Verifying a Replicated Counter Service

Student: _____

Date: 6 October 2026

Implementation: Python, gRPC, Protocol Buffers, pytest

Executive summary

The submitted system evolves a single gRPC counter into a three-replica service with idempotent retries, Lamport logical clocks, majority-acknowledged writes, deterministic failure injection, and automated performance measurement. The final automated suite completed with **11 passed tests** in 10.04 seconds. The required performance check used 2,000 logical requests per configuration and compared a single replica with the three-replica quorum path at 1 and 16 concurrent clients.

11 / 11
automated tests passed

2 / 3
majority required to commit

8,000
logical benchmark requests

1. Requirements under test

Task	Requirement	Verification evidence
A2	Correct Increment/Get behavior, bounded retry, idempotency key, duplicate suppression.	`test_increment_applies_delta`, `test_duplicate_key_not_reapplied`, `test_get_missing_counter`, `test_retry_after_timeout_is_safe`.
A3	Concurrent clients must produce an exact result.	`test_concurrent_increments_exact`: two workers x 1,000 increments produced exactly 2,000.
B1	Lamport update on receive; tick before APPLY/SEND; attach logical time to messages.	Trace-based integration evidence in `logs/b2_trace.log` (28 Lamport events). No separate B1-only pytest was added.
C1	Three independent replicas; commit only at majority; convergence for live replicas.	`test_majority_commit_two_acks`, `test_no_commit_below_majority`, `test_replicas_converge`, plus crash test.

RPC semantics. The mutation is effectively at-most-once for a logical operation while a replica remains alive: retries reuse one idempotency key, and the server checks the deduplication store under the same lock that protects the counter mutation. A duplicate request returns the stored result instead of applying the delta again.

2. Test environment

Item	Value
Operating system	Darwin 27.0.0, RELEASE_ARM64_T8132
Architecture	arm64 (Apple Silicon)
Python	3.14.7
gRPC	grpcio 1.84.0; grpcio-tools 1.84.0
Protocol Buffers	protobuf 7.36.2
Test framework	pytest 9.1.1
Final suite result	**11 passed in 10.04 s**

The complete package capture is included in Appendix B.

3. Test-case table (part 1)

ID	Test	Purpose / expected result	Actual result	Status
T01	test_increment_applies_delta	Increment x by 5; result and Get must equal 5.	new_value=5; duplicate=false; Get=5.	**PASS**
T02	test_duplicate_key_not_reapplied	Same key twice must not apply delta twice.	Second reply returned stored value 5 with duplicate=true; final value 5.	**PASS**
T03	test_get_missing_counter	Unknown counter should not raise.	found=false; value=0.	**PASS**
T04	test_concurrent_increments_exact	Two concurrent workers x 1,000 increments must yield exactly 2,000.	Final value exactly 2,000.	**PASS**
T05	test_retry_after_timeout_is_safe	First request is delayed past deadline; retry uses same key; mutation occurs once.	Retry succeeded; final counter value 1.	**PASS**
T06	test_majority_commit_two_acks	With one replica stopped, 2/3 acknowledgements must still commit.	committed=true; ack_count=2; required=2.	**PASS**

Synchronization and isolation

The server uses a lock around both the deduplication check and counter mutation, preventing check-then-act races. Each replica is a separate server instance with independent in-memory counter and deduplication state. The automated server fixtures use ephemeral ports and shut down their servers after each test, which keeps tests isolated and reproducible.

Retry behavior

Client RPCs use a 2-second deadline. On `DEADLINE_EXCEEDED` or `UNAVAILABLE`, the client performs at most three retries using the same logical idempotency key. The backoff sequence is 0.2 s, 0.4 s, and 0.8 s.

3. Test-case table (part 2)

ID	Test	Purpose / expected result	Actual result	Status
T07	test_no_commit_below_majority	With two replicas stopped, one acknowledgement must not be reported as committed.	committed=false; ack_count=1. Surviving replica contained value 1, but client did not present it as committed.	**PASS**
T08	test_replicas_converge	After a batch of quorum writes, all live replicas should hold identical values.	After five writes, values were [5, 5, 5].	**PASS**
T09	test_replica_crash_mid_request	One replica drops the request after RECV; remaining two acknowledgements must commit and no exception escapes.	Committed with 2 acknowledgements; faulty replica reported UNAVAILABLE; two live replicas held value 1.	**PASS**
T10	test_request_duplication	Duplicate request must be detected and value must move once.	Second response duplicate=true; final value 1.	**PASS**
T11	test_induced_timeout_with_retry	Delay first response by 2500 ms, exceeding the 2-second deadline; retry must be safe.	Retry returned the existing result; duplicate=true; final value 1.	**PASS**

4. Failure-injection results

Scenario	Injected fault	Observed invariant	Evidence
Replica failure mid-request	`fault="drop-after-recv"`	The failed replica returned UNAVAILABLE, but the other two replicas acknowledged and the client committed the write.	`test_replica_crash_mid_request` - PASS.
Request duplication	Same idempotency key sent twice.	Second reply marked duplicate; counter changed exactly once.	`test_request_duplication` - PASS.
Induced timeout + retry	First Increment response delayed 2500 ms.	Client deadline expired, retry reused the same key, final value remained 1.	`test_induced_timeout_with_retry` - PASS.

> **Below-majority anomaly.** A write can fail from the client's point of view while still being applied on one replica. This was observed in T07: only one acknowledgement arrived, so the client correctly returned not committed, but the surviving replica contained the new value. A real consensus protocol prevents an uncommitted entry from being treated as committed and uses an agreed replicated log to reconcile replicas.

Read consistency risk

`Get` reads from one replica. That replica may be stale or may contain a partially applied write, so a single-replica read is not guaranteed to reflect the latest majority-committed state. Quorum reads, leader-based reads, or log replication would provide stronger consistency.

5. Performance check

The benchmark executed 2,000 logical Increment requests for each required configuration. p95 was computed by sorting the recorded latencies and selecting the 95th-percentile order statistic (array index `ceil(0.95*n)-1` in zero-based Python indexing).

Configuration	Median latency (ms)	p95 latency (ms)	Requests
Single replica, 1 client	0.131	0.166	2000
Single replica, 16 clients	2.129	3.840	2000
Quorum (3 replicas), 1 client	0.395	0.442	2000
Quorum (3 replicas), 16 clients	6.791	8.139	2000

The quorum path increased median latency from 0.131 ms to 0.395 ms with one client, about 3.0x, because each logical write produces three RPCs instead of one and must collect replica acknowledgements. At 16 clients, median latency increased from 2.129 ms to 6.791 ms, about 3.2x, while p95 rose from 3.840 ms to 8.139 ms. Concurrency adds queueing and thread scheduling overhead on both the client and servers, and quorum replication amplifies that work. For a small counter service these absolute local-machine latencies remain low, but the cost would be more significant across real networks. The current implementation also waits for all launched replica futures before returning, so an early-return optimization after the second acknowledgement could reduce quorum latency.

6. Task B2 ordering analysis

```
Causal chain 1:
client-3 SEND Get(x) L=1
  -> replica-A RECV Get(x) L=2
  -> replica-A SEND GetReply L=3
  -> client-3 RECV GetReply L=4

Causal chain 2:
replica-A APPLY y->1 L=5
  -> replica-A SEND IncrementReply L=6
  -> client-2 RECV IncrementReply L=7
  -> client-2 SEND second Increment(y) L=8
  -> replica-A RECV second Increment(y) L=12
```

Both chains are proven by a combination of local process order and message send/receive edges. Therefore the earlier event in each chain happens-before the later event.

Concurrent pair with different Lamport values: `client-3 SEND Get(y) L=5` and `client-2 SEND second Increment(y) L=8`. There is no message or local-order path from one client event to the other, so they are concurrent even though 5 < 8. Lamport clocks guarantee that happened-before implies a smaller timestamp, but the converse is false.

Vector-clock limitation: Lamport values alone cannot tell whether two differently numbered events are concurrent. Vector clocks retain per-process causal history and can distinguish causal order from concurrency directly.

7. Known limitations

1. **In-memory durability only.** Counter and deduplication state are lost when a replica process restarts. A restarted replica therefore cannot preserve previously committed data without additional persistence and repair.
2. **No consensus or log repair.** Replicas accept writes independently. A below-majority write can partially apply, and there is no leader, term, commit index, or replicated log to reconcile it later.
3. **Single-replica reads may be stale.** Get can return a value from a replica that has not observed the latest committed write.
4. **Quorum client waits for all launched futures.** The write is classified by majority, but the current implementation does not return immediately after the second acknowledgement; a slow third replica can add latency.

8. Reflection questions

1) Why is a timeout ambiguous, and why are retries safe here?

After a timeout, the client cannot know whether the request was lost, the server executed it but the reply was lost, or the server is simply slow. The client therefore retries with the same idempotency key. Because the server stores the result under that key and checks the store under the same lock as the mutation, a retry returns the stored result instead of applying the delta again.

2) Why does a smaller Lamport value not prove that an event happened first?

Lamport clocks provide one direction only: if event a happens-before event b, then L(a) < L(b). Two concurrent events can still receive different Lamport values because each process increments its own counter independently. In the captured trace, client-3 SEND Get(y) at L=5 and client-2 SEND the second Increment(y) at L=8 are concurrent despite the numeric ordering.

3) What failures can a 3-replica majority tolerate?

With three replicas, one unavailable replica still leaves two acknowledgements, so quorum write liveness is preserved and committed writes are acknowledged by a majority. If two replicas are unavailable, the client cannot reach a majority, so liveness is lost and the write must not be reported as committed. The client-side rule preserves the safety property of not claiming below-majority success, although the simplified system can still leave a partial uncommitted write on a surviving replica.

4) Smallest change needed to survive replica restarts

The smallest practical change is to persist counter state, deduplication keys, or preferably an operation log to durable storage before acknowledging a write. This anticipates the durable replicated-log mechanism used by consensus protocols such as Raft, where committed log entries can be recovered and replayed after a restart.

9. Conclusion

The implementation satisfies the core assignment behaviors demonstrated by the automated suite: exact counter mutation, duplicate suppression, concurrency safety, timeout-safe retry, majority commit, below-majority failure reporting, replica convergence, and deterministic fault handling. The performance results quantify the expected cost of quorum replication, and the Lamport trace demonstrates causal reasoning without assuming synchronized wall clocks. The remaining limitations are deliberate simplifications of the assignment and point directly toward persistence, stronger read consistency, and consensus-based replication.

Appendix A - Task B2 trace (28 events)

This is the captured event-log excerpt used for the ordering analysis. It is sorted by Lamport value for review; equal timestamps on different processes do not establish causal order.

```
[client-1] SEND Increment(counter=x, delta=1) L=1
[client-2] SEND Increment(counter=y, delta=1) L=1
[client-3] SEND Get(counter=x) L=1
[replica-A] RECV Get(counter=x) L=2 (received L=1)
[replica-A] SEND GetReply(value=0, found=False) L=3
[client-3] RECV GetReply(value=0, found=False) L=4 (received L=3)
[replica-A] RECV Increment(counter=y, delta=1) L=4 (received L=1)
[client-3] SEND Get(counter=y) L=5
[replica-A] APPLY counter=y -> 1 L=5
[replica-A] SEND IncrementReply(new_value=1) L=6
[client-2] RECV IncrementReply(new_value=1) L=7 (received L=6)
[replica-A] RECV Increment(counter=x, delta=1) L=7 (received L=1)
[client-2] SEND Increment(counter=y, delta=1) L=8
[replica-A] APPLY counter=x -> 1 L=8
[replica-A] SEND IncrementReply(new_value=1) L=9
[client-1] RECV IncrementReply(new_value=1) L=10 (received L=9)
[replica-A] RECV Get(counter=y) L=10 (received L=5)
[client-1] SEND Increment(counter=x, delta=1) L=11
[replica-A] SEND GetReply(value=1, found=True) L=11
[client-3] RECV GetReply(value=1, found=True) L=12 (received L=11)
[replica-A] RECV Increment(counter=y, delta=1) L=12 (received L=8)
[replica-A] APPLY counter=y -> 2 L=13
[replica-A] SEND IncrementReply(new_value=2) L=14
[client-2] RECV IncrementReply(new_value=2) L=15 (received L=14)
[replica-A] RECV Increment(counter=x, delta=1) L=15 (received L=11)
[replica-A] APPLY counter=x -> 2 L=16
[replica-A] SEND IncrementReply(new_value=2) L=17
[client-1] RECV IncrementReply(new_value=2) L=18 (received L=17)
```

Appendix B - Package versions

```
Python 3.14.7

grpcio==1.84.0
grpcio-tools==1.84.0
iniconfig==2.3.0
packaging==26.3
pluggy==1.6.0
protobuf==7.36.2
Pygments==2.21.0
pytest==9.1.1
setuptools==84.0.0
typing_extensions==4.16.0
```

Appendix C - Reproduction evidence

```
Final automated suite:
.......... [100%]
11 passed in 10.04s

Benchmark command:
python3 perf_benchmark.py

Test command:
python3 -m pytest -q

Captured trace:
logs/b2_trace.log
```

Incremental implementation history

The repository was developed incrementally across communication, Lamport-clock instrumentation, replication/quorum behavior, automated testing, failure injection, performance benchmarking, and reproduction instructions. The working tree was clean after the final environment capture.

Submission checklist

Artifact	Status
counter.proto + generated stubs	Present
server.py / client.py / clocks.py	Present
tests/test_counter.py	Present
tests/test_failures.py	Present
perf_benchmark.py	Present
logs/b2_trace.log	Present
README.md	Present
report.pdf	Generated